

How this works: webhooks, schedules, credentials, and when I use Zapier vs n8n

This is the vocabulary behind the eight workflows in this repo, explained with the systems I actually run for clients. If you only read one file here, read this one, then run `bash demo/demo.sh` and watch it happen.

The shape of every automation

Every automation I have built has the same four parts:

1. **A trigger.** Something happens outside: a form is submitted, a phone call ends, a customer texts, the clock hits 10:00.
2. **A transform.** The event arrives in whatever shape the sender uses. I turn it into the shape the next step needs (normalize the phone number, pull the fields I care about, decide which branch to take).
3. **An action.** Call an API: place a phone call, write a note on a CRM contact, send a text, send an email.
4. **A record.** Something a person can look at later: a tag on the contact, a line in a log, an email in the owner's inbox.

The two trigger types that matter are webhooks and schedules.

Webhooks

A webhook is an HTTP request that another system sends to a URL I own the moment something happens. Instead of my system asking "anything new?" every minute (polling), the other system tells me.

Concrete examples from this repo:

- **GoHighLevel** (a CRM) has a "Webhook" step in its workflow builder. When a new lead is created, it POSTs the contact as JSON to my URL. Workflow 01 receives that and places an AI call within seconds. That is the whole point of "speed to lead": the call happens while the person is still looking at the website.
- **Vapi** (the voice AI) POSTs an "end-of-call report" when a call ends, with a transcript summary and structured fields the assistant extracted (did they book, what service, what vehicle). Workflow 02 turns that into CRM tags.
- **Twilio** POSTs a "status callback" after every phone call (answered, no-answer, busy) and a message webhook for every inbound text. Workflow 03 uses both: one to catch missed calls, one to record STOP.

Things I always do with an inbound webhook:

- **Verify it came from who it claims.** Anyone on the internet can POST to a URL. I put a shared secret in a header and reject requests without it (workflow 01, "Verify shared secret"). Twilio and Stripe sign their requests instead; same idea, stronger.
- **Respond 200 quickly, even when I skip the event.** If I return an error, the sender retries, and now I have the same lead twice. In workflow 01 a duplicate or bad phone number still gets a 200 with `{"skipped": "..."}.`
- **Expect two payload shapes.** The CRM sends `first_name`, the web form sends `name`. The first Code node normalizes both into one object so nothing downstream cares where the lead came from.

In n8n the trigger is the **Webhook node**. Its production URL only exists while the workflow is active (published). During development you use the test URL and click "Listen for test event".

Schedules (cron)

A schedule trigger runs a workflow on a clock instead of on an event. "Cron" is the old Unix name for that, and `0 10 * * 1-5` is the cron expression for "10:00 every Monday to Friday" (minute, hour, day of month, month, day of

week).

Workflow 04 runs on that schedule. It asks the CRM for every contact tagged `ai-no-answer`, decides per contact whether to try again, and paces the calls a few seconds apart so the phone system is not hit with twenty calls at once.

Why a schedule and not a chain of "wait 1 day" steps after the first call? Two reasons I learned the hard way:

- A wait step means the execution is sitting there for days. If the instance restarts or someone edits the workflow, those pending executions are gone or wrong.
- With a daily pass, the **state lives in the CRM** as tags (`ai-attempt-2`, `ai-booked`, `ai-do-not-call`). The owner can see it, the owner can change it (tag someone `ai-do-not-call` by hand and the job respects it tomorrow), and there is no second database that can drift from what the CRM shows.

The demo shows this: Dana books during the end-of-call step, and when the follow-up pass runs afterward she is skipped as `closed`. The job never had to be told.

In n8n this is the **Schedule Trigger node**. I also put a Manual Trigger next to it so you can run the same workflow by hand with the "Execute workflow" button, or from the command line with `n8n execute --id ...`, which is what the demo does.

Credentials

A credential is a secret that proves to an outside API that a request is mine: an API key, a bearer token, a username and password, an OAuth token. Every real API rejects requests without one (the mock servers in `demo/` do too).

Rules I follow, and the reason for each:

- **Secrets never go in the workflow file.** The JSON in `workflows/` references credentials by name ("Vapi API key", "GoHighLevel API token"). The actual values live in n8n's credential store, which encrypts them at rest with the instance's encryption key. That means I can put the workflows on GitHub, and it means a leaked export does not leak keys.
- **One credential per service, shared by every node that talks to it.** Rotate the key in one place.
- **Least privilege.** My Gmail lookup tool asks for the `gmail.metadata` scope only, so even a compromised token cannot read a message body. Same thinking for a CRM token: a location-scoped token, not an agency-wide one.
- **The secret I check on inbound webhooks is a credential too**, just one I hand to the sender instead of the other way round.

In n8n, credentials are created under Settings, attached to a node from a dropdown, and the HTTP Request node has "Generic credential type" (header auth, basic auth, OAuth2) for APIs that do not have a dedicated node. The demo imports six credentials with `n8n import:credentials` and they are encrypted on the way in.

Zapier vs n8n (vs writing code)

I have shipped all three. They are not competitors so much as three points on a line, and the question is who has to maintain it, how many events run through it, and where the data is allowed to go.

	Zapier	n8n	Custom code (my Node.js services)
Who can read and change it	The business owner. It is a list of plain-English steps.	An engineer, or a technical ops person. Visual, but branching and expressions need some literacy.	Only me.
Best at	Two to five step linear flows between well-known SaaS apps.	Branching, loops, retries, code steps, error handling, dozens of steps, self-hosted.	Anything with tight latency, in-process logic, or a shape no node fits.

	Zapier	n8n	Custom code (my Node.js services)
Cost model	Per task. Fine at 100 events a month, painful at 100,000.	Flat (self-hosted) or per execution (cloud), and one execution can run many steps.	Hosting only.
Where the data goes	Zapier's cloud.	Your server if self-hosted.	Your server.
Auditability	Task history, per Zap.	Full execution log with every node's input and output, retryable, exportable.	Whatever logging I wrote.

How I decided in practice:

- **fieldd CRM sync (300 Mobile Detailing): Zapier.** fieldd has a Zapier app and an undocumented REST API. Volume is a handful of leads a day. The owner is not technical and wanted to see the integration himself. A Zapier catch hook in and a Zap out means he can open it, read it, and turn it off. Building on the undocumented API would have broken silently the day they changed it.
- **Speed-to-lead and follow-ups (3Diamond): custom Node.js on Vercel, and now n8n.** Sub-second response to a webhook, retries, a daily cron, state in CRM tags, an error path. That is more logic than a Zap should hold, and the per-task pricing would not make sense once the shop scales its ads.
- **A regulated environment (the reason I rebuilt these in n8n).** Self-hosted n8n keeps every payload inside the company's network, gives a full execution log for audit, lets credentials sit in an encrypted store the security team controls, and still lets a non-developer read the flow. That is the trade a bank or a GSE needs, and Zapier cannot offer it.

Rule of thumb: if the owner has to be able to change it and it is short, Zapier. If it has branches, retries, or volume, n8n. If it has to be fast or the logic is genuinely unusual, code, and often n8n calling that code over HTTP.

Two n8n-specific things worth knowing

Workflow static data (`$getWorkflowStaticData`) is a small per-workflow store n8n persists between executions. I use it for the six-hour dedupe, the once-a-day text guard, and the STOP list. It is read at the start of an execution and written at the end, so two executions a few milliseconds apart can both read the old value. For a single shop's phone line that is fine. For thousands of events a minute I would use a database or n8n's Data Tables and a proper uniqueness key.

Error workflows. Any workflow can name another workflow as its error handler (Settings, "Error workflow"). n8n runs it with the failing workflow, node and message as input. Workflow 05 is that handler for the others. It throttles the same failure to one alert an hour so a flapping API does not fill the inbox. The demo triggers it by making the mock Vapi return 500 for one phone number.